

Suvenīri

Amaru pērk suvenīrus ārzemju veikalā. Ir N **tipu** suvenīri. Veikalā ir pieejams bezgalīgs skaits katra tipa suvenīru.

Katram suvenīra tipam ir noteikta nemainīga cena. Tas ir, i -tā tipa suvenīra ($0 \leq i < N$) cena ir $P[i]$ monētas, kur $P[i]$ ir naturāls skaitlis.

Amaru zina, ka suvenīru tipi ir sakārtoti dilstošā secībā pēc cenas un ka suvenīru cenas ir atšķirīgas. Precīzāk sakot, $P[0] > P[1] > \dots > P[N-1] > 0$. Turklāt, viņam izdevās noskaidrot $P[0]$ vērtību. Diemžēl Amaru nav nekādas citas informācijas par suvenīru cenām.

Lai nopirktu suvenīrus, Amaru ar pārdevēju veiks vairākus darījumus.

Katram darījumam ir šādi soļi:

1. Amaru iedod kādu (naturālu) monētu skaitu pārdevējam.
2. Pārdevējs noliek šīs monētas kaudzītē uz galda telpā veikala aizmugurē, kur Amaru tās neredz.
3. Pārdevējs pa vienam apsver katru suvenīru tipu $0, 1, \dots, N-1$ (tādā secībā). Katru tipu katrā darījumā apsver **tieši vienreiz**.
 - Kad tiek apsvērts suvenīru tips i , ja šobrīd monētu skaits kaudzītē ir vismaz $P[i]$, tad
 - pārdevējs izņem $P[i]$ monētas no kaudzītes un
 - noliek vienu i -tā tipa suvenīru uz galda.
4. Pārdevējs iedod Amaru visas monētas, kas atlikušas kaudzītē, un visus uz galda esošos suvenīrus.

Ņemiet vērā, ka pirms darījuma sākuma uz galda nav nevienas monētas vai suvenīra.

Jūsu uzdevums ir sniegt norādījumus Amaru, lai viņš veiktu darījumus tā, ka:

- katrā darījumā viņš nopērk **vismaz vienu** suvenīru un
- kopumā viņš nopērk **tieši** i suvenīrus ar tipu i katram i , kur $0 \leq i < N$. Ņemiet vērā, ka tas nozīmē, ka Amaru nevajadzētu pirkt nevienu 0. tipa suvenīru.

Amaru nav nepieciešams minimizēt darījumu skaitu, un viņam ir pieejams neierobežots monētu skaits.

Implementācijas detaļas

Jums jāimplementē šāda procedūra.

```
void buy_souvenirs(int N, long long P0)
```

- N : suvenīru tipu skaits.
- $P0$: $P[0]$ vērtība.
- Šī procedūra katram testam tiek izsaukta tieši vienreiz.

Iepriekš minētā procedūra, lai dotu Amaru norādījumus, kā veikt darījumu, var veikt šādas procedūras izsaukumus:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : monētu, ko Amaru iedod pārdevējam, skaits.
- Procedūra atgriež pāri. Pirmais pāra elements ir masīvs L , kas satur nopirkto suvenīru tipus (pieaugošā secībā). Otrais elements ir vesels skaitlis R – pēc darījuma Amaru atgriezto monētu skaits.
- Jāizpildās $P[0] > M \geq P[N - 1]$. Nosacījums $P[0] > M$ nodrošina, ka Amaru nenopērk nevienu 0. tipa suvenīru, un $M \geq P[N - 1]$ nodrošina, ka Amaru nopērk vismaz vienu suvenīru. Ja šie nosacījumi netiek ievēroti, Jūsu risinājums saņems vērtējumu `Output isn't correct: Invalid argument` (izvaddati nav pareizi: nederīgs arguments). Ņemiet vērā, ka atšķirībā no $P[0]$ vērtības $P[N - 1]$ vērtība ievaddatos nav dota.
- Procedūra var tikt izsaukta ne vairāk kā 5000 reizes katram testam.

Vērtētāja uzvedība **nav adaptīva**. Tas nozīmē, ka cenu virkne P tiek fiksēta pirms procedūra `buy_souvenirs` tiek izsaukta.

Ierobežojumi

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ katram i , kur $0 \leq i < N$.
- $P[i] > P[i + 1]$ katram i , kur $0 \leq i < N - 1$.

Apakšuzdevumi

Apakšuzdevums	Punkti	Papildu ierobežojumi
1	4	$N = 2$
2	3	$P[i] = N - i$ katram i , kur $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ katram i , kur $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ katram i , kur $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ katram i , kur $0 \leq i < N - 1$.
6	33	Bez papildu ierobežojumiem.

Piemērs

Aplūkosim šādu izsaukumu.

```
buy_souvenirs(3, 4)
```

Ir $N = 3$ suvenīru tipi, un $P[0] = 4$. Var ievērot, ka ir iespējamās tikai trīs cenu virknes P : $[4, 3, 2]$, $[4, 3, 1]$, un $[4, 2, 1]$.

Pieņemsim, ka `buy_souvenirs` izsauc `transaction(2)`. Pieņemsim, ka izsaukums atgriež $([2], 1)$, kas nozīmē, ka Amaru nopirka vienu 2. tipa suvenīru un pārdevējs viņam iedeva atpakaļ 1 monētu. No šīs informācijas var izsecināt, ka $P = [4, 3, 1]$, jo:

- Ja būtu $P = [4, 3, 2]$, `transaction(2)` atgrieztu $([2], 0)$.
- Ja būtu $P = [4, 2, 1]$, `transaction(2)` atgrieztu $([1], 0)$.

Tad `buy_souvenirs` var izsaukt `transaction(3)`, kas atgriež $([1], 0)$, kas nozīmē, ka Amaru nopirka vienu 1. tipa suvenīru un pārdevējs viņam atdeva atpakaļ 0 monētas. Līdz šim viņš kopumā ir nopircis vienu 1. tipa suvenīru un vienu 2. tipa suvenīru.

Visbeidzot, `buy_souvenirs` var izsaukt `transaction(1)`, kas atgriež $([2], 0)$, kas nozīmē, ka Amaru nopirka vienu 2. tipa suvenīru. Ņemiet vērā, ka šeit būtu bijis iespējams izmantot arī `transaction(2)`. Šobrīd Amaru kopumā ir nopircis vienu 1. tipa un divus 2. tipa suvenīrus, kas atbilst prasībām.

Paraugvērtētājs

Ievaddatu formāts:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Izvaddatu formāts:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Šeit $Q[i]$ ir kopumā nopirkto i -tā tipa suvenīru skaits katram i , kur $0 \leq i < N$.